

# GÖZTEPE MESLEKİ VE TEKNİK ANADOLU LİSESİ

BİLİŞİM TEKNOLOJİLERİ ALANI

## BİLET-GEÇ

Web Tabanlı Etkinlik Bilet Yönetim Sistemi

"Web Tabanlı Uygulama" ve "Seçmeli Yazılım Projesi" Dersleri

**Dönem Sonu Bitirme Projesi Raporu**

**Öğrenci Adı Soyadı:** Muhittin Ali Akyüz

**Okul Numarası:** 992

**Sınıfı / Şubesi:** 11 / ATP

## 2. ÖNSÖZ (KİŞİSEL DEĞERLENDİRME)

---

Bu rapor, lise 11. sınıf ATP şubesi kapsamında geliştirilen "Bilet-Geç" adlı web tabanlı etkinlik ve bilet yönetim sisteminin teknik belgesidir. Dönem boyunca aldığım Web Tabanlı Uygulama ve Seçmeli Yazılım Projesi dersleri, bana hem teorik bilgileri pekiştirme hem de gerçek dünyada karşılaşılan karmaşık algoritmalara pratik çözümler üretme fırsatı tanıdı.

Geliştirme süreci boyunca sunucu tarafında PHP ve MySQL ilişkisel veritabanı yönetim sistemleri üzerinde derinlemesine deneyim kazandım. PDO (PHP Data Objects) yapısını kullanarak güvenli veritabanı katmanları kurgulamayı, prepared statements (hazırlanmış ifadeler) ile SQL Injection açıklarını tamamen engellemeyi ve session mimarisiyle güvenli kullanıcı oturumları yönetmeyi öğrendim. Ayrıca Cross-Site Request Forgery (CSRF) saldırılarına karşı token tabanlı savunma mekanizmaları geliştirmek, yazılımda güvenlik vizyonumu genişletti.

Arayüz mimarisinde ise harici hantal kütüphaneler yerine tamamen vanilla CSS ve modern dizayn yaklaşımları uygulayarak premium koyu tema (dark mode) tasarımı oluşturdum. Design token mantığı ile merkezi renk yönetimini kavradım. Bu süreçte algoritmik düşünme becerimi en çok geliştiren modüller ise; zamanlayıcı rezervasyon iptal mekanizması, etkinlik iptallerinde geçmiş tarihli satılmış biletlerin iade edilmesini engelleyen kontrol mantığı ve firmaların yalnızca kendi gelir/bilet verilerini görmelerini sağlayan yetki izolasyonu oldu. Proje, bütünsel bir web uygulamasının mimari planlamasından canlıya alınışına kadar olan tüm döngüyü kavramamı sağlayan en büyük kazanımım olmuştur.

## 3. İÇİNDEKİLER

---

1. Kapak Sayfası
2. Önsöz (Kişisel Değerlendirme)
3. İçindekiler
4. Proje Tanımı ve Amacı
  - 4.1 Giriş ve Sistem İhtiyacı
  - 4.2 Çözülen Problemler ve Senaryolar
  - 4.3 Hedef Kitle ve Kullanıcı Roller
  - 4.4 Projenin Kapsamı ve Sınırları
5. Kullanılan Teknolojiler ve Araçlar
  - 5.1 Önyüz (Front-End) Detayları
  - 5.2 Arkayüz (Back-End) ve Sunucu Mimarisi
  - 5.3 Veritabanı Yönetim Sistemi
6. Problemin Çözümünde İzlenen Yol (Sistem Analizi)
  - 6.1 Detaylı Veritabanı İlişkisel Yapısı (E-R Şeması Analizi)
  - 6.2 Yazılımsal Güvenlik ve Yetkilendirme Standartları
  - 6.3 İş Mantığı (Etkinlik İptal ve İade Algoritması)
  - 6.4 Rezervasyon Süresi ve Kota Yönetimi
7. Veritabanı Veri Tanımlama Dili (DDL) Tablo Yapıları
  - 7.1 users Tablosu Şeması
  - 7.2 categories Tablosu Şeması
  - 7.3 events Tablosu Şeması
  - 7.4 tickets Tablosu Şeması
  - 7.5 reservations Tablosu Şeması
  - 7.6 refunds Tablosu Şeması
8. Proje Yapım Aşamaları ve Geliştirme Süreci
  - 8.1 1. Aşama: Gereksinim Analizi ve Planlama
  - 8.2 2. Aşama: Veritabanı Tasarımı (Schema Design & DDL)
  - 8.3 3. Aşama: Arayüz (Front-End) Tasarımı ve CSS Mimarisi
  - 8.4 4. Aşama: Sunucu (Back-End) Programlama ve OOP Entegrasyonu
  - 8.5 5. Aşama: Güvenlik, Hata Ayıklama (Debugging) ve Refactoring
  - 8.6 6. Aşama: Testler, Doğrulama ve Canlı Ortam Simülasyonu

## 9. Proje Modüllerinin Kod Yapısı ve Analizi

9.1 Güvenlik ve Yardımcı Fonksiyonlar (includes/functions.php)

9.2 Oturum ve Yetkilendirme Mantiğı (includes/auth.php)

9.3 İade İş Mantiğı Fonksiyonları (includes/refund\_functions.php)

9.4 İstemci Arayüzü - Etkinlik Arama ve Listeleme (events.php)

9.5 Bilet Satış ve Simüle POS Ödeme Ekranı (event.php)

9.6 Yönetici Paneli İade Talebi API Yapısı (admin/api/manage\_refunds.php)

9.7 Yönetici İade Kontrol Sayfası Arayüzü (admin/manage\_refunds.php)

9.8 Kullanıcı Profil ve Bilet Yönetim Sayfası (my\_tickets.php)

## 10. Karşılaşılan Zorluklar ve Çözümleri

10.1 Eşzamanlı Satışlarda Yarış Durumu (Race Condition)

10.2 Arayüz Grafik Performansı ve GPU Optimizasyonu

10.3 Veri Güvenliği ve İzolasyon Zafiyetleri

## 11. Proje Ekran Görüntüleri ve Kullanım Adımları

## 12. Kaynakça



## 4. PROJE TANIMI VE AMACI

---

### 4.1 Giriş ve Sistem İhtiyacı

Günümüzde kültürel ve sportif etkinliklerin dijitalleşmesi, bilet satış süreçlerinin hızlı ve güvenli bir şekilde internet ortamına taşınmasını zorunlu kılmıştır. Geleneksel gişe biletleme sistemlerinin yerini alan dijital biletleme servisleri, son kullanıcıların etkinliklerden haberdar olmalarını, anlık koltuk veya kapasite durumunu görerek güvenle ödeme yapmalarını kolaylaştırmaktadır. Bilet-Geç, kullanıcıların fiziksel sıralarda beklemeden, saniyeler içinde istedikleri etkinliğe yer ayırtıp bilet satın almalarını amaçlayan, modern tasarım standartlarına uygun geliştirilmiş web tabanlı bir biletleme otomasyonudur.

### 4.2 Çözülen Problemler ve Senaryolar

- **Kapasite Aşımı (Overbooking):** Popüler etkinliklerde, son kalan biletlerin birden fazla kişiye aynı anda satılması problemi, veritabanı düzeyinde uygulanan satır kilitleme (Row Locking) yöntemleriyle çözülmüştür.
- **Sahte Bilet Üretimi:** Her bilet satışı için sistem tarafından benzersiz bir kriptografik order\_code üretilir. Bu kodlar etkinlik girişinde doğrulanabilir altyapıya sahiptir.
- **Etkinlik İptal Mağduriyeti:** Organizatör tarafından iptal edilen etkinliklerde, kullanıcıların ödemiş oldukları bilet tutarları, insan müdahalesine gerek kalmaksızın sistem tarafından otomatik iade talebine dönüştürülür.
- **Firma Yetki İzolasyonu:** Bir firmanın, sistemdeki başka firmalara ait etkinlikleri veya bilet satış rakamlarını görüntülemesini engellemek için filtreleme katmanları geliştirilmiştir.

### 4.3 Hedef Kitle ve Kullanıcı Roller

Sistem, yetki düzeylerine göre 3 farklı kullanıcı rolünü barındırır:

1. **Müşteriler (Customers):** Arayüze girerek etkinlik arar, bilet alır ve bilet geçmişini yönetir.
2. **Firmalar (Partners):** Kendi etkinliklerini tanımlar, fiyat belirler, kapasite ayarlar ve kendi etkinliklerinin bilet iadelerini onaylar.
3. **Süper Admin (Admins):** Sistemdeki tüm kullanıcıları, firmaları ve etkinlikleri yönetir. Sistem genelindeki toplam hasılat ve ciro grafiklerini görür ve etkinliklerin sahipliğini değiştirebilir.

#### **4.4 Projenin Kapsamı ve Sınırları**

Sanal POS entegrasyonu gerçek banka API'leri yerine güvenli bir simülasyon üzerinden yürütülmektedir. Proje tamamen responsive (duyarlı) mobil tasarım standartlarına uygundur ve ek bir uygulama indirmeye gerek kalmadan tüm tarayıcılarda çalışır.

## 5. KULLANILAN TEKNOLOJİLER VE ARAÇLAR

---

### 5.1 Önyüz (Front-End) Detayları

- **Semantic HTML5:** SEO kurallarına uygun, <header>, <nav>, <main>, <section>, <footer> gibi modern yerleşim elementleri kullanılmıştır.
- **Modern CSS3 & Design Tokens:** Harici framework kullanılmadan geliştirilen CSS dosyasında, tema renkleri :root altında toplanmıştır. backdrop-filter gibi özelliklerle modern cam tasarımı (glassmorphic) elde edilmiştir.
- **Native JavaScript (ES6+):** Sayfa yenilenmeden çalışan dinamik Fetch API çağrıları, form doğrulama işlemleri ve mobil hamburger menü açılışları vanilla JS ile kodlanmıştır.

### 5.2 Arkayüz (Back-End) ve Sunucu Mimarisi

- **PHP 8.x:** Performans, modüler yapı ve OOP (Nesne Yönelimli Programlama) standartlarına uygun PHP kodu yazılmıştır. Sayfa geçişlerinde oturum verilerini kaybetmemek için güvenli PHP Session mekanizması kurulmuştur.
- **Model-View-Controller (MVC) Yaklaşımı:** İş mantığı (PHP Controller), arayüz tasarımı (View) ve veritabanı işlemleri (Model) dosyaları mimari açıdan birbirinden ayrıştırılmıştır.

### 5.3 Veritabanı Yönetim Sistemi

- **MySQL:** İlişkisel veritabanı motoru olarak InnoDB kullanılmıştır. Bu motor, transaction işlemlerini desteklemesi ve yabancı anahtarlarla veri bütünlüğünü koruması sebebiyle tercih edilmiştir.
- **PDO (PHP Data Objects):** Veritabanı sürücülerinden bağımsız, SQL Injection zafiyetlerine karşı korumalı prepare ve bindValue metotlarını barındıran PHP veri objeleri kullanılmıştır.



## 6. PROBLEMİN ÇÖZÜMÜNDE İZLENEN YOL (SİSTEM ANALİZİ)

### 6.1 Detaylı Veritabanı İlişkisel Yapısı (E-R Şeması Analizi)

Tablolar arasındaki bütünlüğü korumak için ON DELETE CASCADE kuralları eklenmiştir.



## 6.2 Yazılımsal Güvenlik ve Yetkilendirme Standartları

Sistemde kullanıcı güvenliği ve form bütünlüğü CSRF ve Session tabanlı kontrollerle sağlanır. Güvenli olmayan veri girişleri PHP'nin htmlspecialchars() fonksiyonuyla filtrelenerek XSS (Cross-Site Scripting) engellenir.



### 6.3 İş Mantığı (Etkinlik İptal ve İade Algoritması)

Etkinlik yöneticileri bir etkinliği iptal ettiğinde aşağıdaki kural işletilir:

- Etkinlik tarihi **geçmişse** (etkinlik bittiyse), satılmış biletler iade edilmez.
- Etkinlik tarihi **gelecekteyse**, tüm biletler iptal edilip iade kaydı oluşturulur.

### 6.4 Rezervasyon Süresi ve Kota Yönetimi

Kullanıcı bilet seçtiğinde veritabanına eklenen rezervasyon kaydı `expires_at` süresiyle kontrol edilir. Süresi dolan rezervasyonlar sistem tarafından periyodik olarak silinip etkinliklerin kapasitesi geri iade edilir.

## 7. VERİTABANI VERİ TANIMLAMA DİLİ (DDL) TABLO YAPILARI

---

Sistemin kurulumunda kullanılan MySQL DDL şemaları aşağıda detaylandırılmıştır:

### 7.1 users Tablosu Şeması

Kullanıcı, firma ortağı ve yöneticilerin kimlik ve yetki bilgilerini barındırır.

```
CREATE TABLE `users` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(100) NOT NULL,
  `email` VARCHAR(150) NOT NULL UNIQUE,
  `password_hash` VARCHAR(255) NOT NULL,
  `role` ENUM('customer', 'partner', 'admin') NOT NULL DEFAULT
'customer',
  `loyalty_points` INT DEFAULT 0,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

### 7.2 categories Tablosu Şeması

Etkinliklerin kategorize edilmesini sağlayan tablodur.

```
CREATE TABLE `categories` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `name` VARCHAR(100) NOT NULL,
  `icon` VARCHAR(10) DEFAULT ' '
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

### 7.3 events Tablosu Şeması

Etkinlik detayları, kapasite durumları ve firmalara ait sahiplik ilişkilerini barındırır.

```
CREATE TABLE `events` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `title` VARCHAR(200) NOT NULL,
  `description` TEXT NOT NULL,
  `category_id` INT NOT NULL,
  `total_capacity` INT NOT NULL,
  `remaining_capacity` INT NOT NULL,
  `price` DECIMAL(10,2) NOT NULL,
  `event_date` DATETIME NOT NULL,
  `created_by` INT DEFAULT NULL,
  `status` ENUM('active', 'cancelled', 'completed') DEFAULT 'active',
  `image_url` VARCHAR(255) DEFAULT NULL,
  `city` VARCHAR(100) NOT NULL,
  `venue` VARCHAR(150) NOT NULL,
  FOREIGN KEY (`category_id`) REFERENCES `categories`(`id`) ON DELETE RESTRICT,
  FOREIGN KEY (`created_by`) REFERENCES `users`(`id`) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

## 7.4 tickets Tablosu Şeması

Kullanıcıların satın aldığı biletleri ve benzersiz bilet kodlarını içerir.

```
CREATE TABLE `tickets` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `order_code` VARCHAR(50) NOT NULL UNIQUE,
  `user_id` INT NOT NULL,
  `event_id` INT NOT NULL,
  `price_paid` DECIMAL(10,2) NOT NULL,
  `status` ENUM('active', 'cancelled', 'refunded') DEFAULT 'active',
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON DELETE CASCADE,
  FOREIGN KEY (`event_id`) REFERENCES `events`(`id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

## 7.5 reservations Tablosu Şeması

Satın alma öncesi sepette kilitlenen biletlerin sürelerini tutar.

```
CREATE TABLE `reservations` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `user_id` INT NOT NULL,
  `event_id` INT NOT NULL,
  `ticket_count` INT NOT NULL,
  `expires_at` DATETIME NOT NULL,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON DELETE CASCADE,
  FOREIGN KEY (`event_id`) REFERENCES `events`(`id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

## 7.6 refunds Tablosu Şeması

İade edilen biletlerin süreç takip tablosudur.

```
CREATE TABLE `refunds` (  
  `id` INT AUTO_INCREMENT PRIMARY KEY,  
  `ticket_id` INT NOT NULL,  
  `user_id` INT NOT NULL,  
  `event_id` INT NOT NULL,  
  `refund_amount` DECIMAL(10,2) NOT NULL,  
  `status` ENUM('pending', 'approved', 'completed', 'rejected') DEFAULT  
  'pending',  
  `rejection_reason` TEXT DEFAULT NULL,  
  `requested_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (`ticket_id`) REFERENCES `tickets`(`id`) ON DELETE CASCADE,  
  FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON DELETE CASCADE,  
  FOREIGN KEY (`event_id`) REFERENCES `events`(`id`) ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```



## 8. PROJE YAPIM AŞAMALARI VE GELİŞTİRME SÜRECİ

Bilet-Geç projesinin fikir aşamasından nihai ürüne dönüştürülmesine kadar geçen süreçte sistematik bir yazılım geliştirme yaşam döngüsü (SDLC) izlenmiştir. Yapım aşamaları sırasıyla şu şekildedir:

### 8.1 1. Aşama: Gereksinim Analizi ve Planlama

- **Fikir Belirleme:** İlk aşamada, piyasadaki mevcut bilet satış platformlarının zayıf yönleri (overbooking, kötü kullanıcı deneyimi, firmalar için karmaşık raporlama) analiz edilmiştir.
- **Gereksinim Belirleme:** Müşteri, organizatör firma ve sistem yöneticisinin ihtiyaç duyacağı ekranlar ve iş kuralları listelenmiştir.
- **Teknoloji Seçimi:** Düşük sunucu maliyeti ve yüksek taşınabilirlik hedeflenerek ek kütüphane gerektirmeyen PHP ve MySQL ikilisi seçilmiştir.

### 8.2 2. Aşama: Veritabanı Tasarımı (Schema Design & DDL)

- **İlişkisel Modelleme:** Etkinliklerin, biletlerin, iadelerin ve geçici koltuk rezervasyonlarının birbiriyle olan ilişkileri normalizasyon kurallarına (3NF) uygun şekilde tasarlanmıştır.
- **Bütünlük Kontrolleri:** Etkinlik silindiğinde veya üye silindiğinde sistemde artık veri (orphan records) kalmaması amacıyla veritabanı düzeyinde yabancı anahtar kısıtlamaları (FOREIGN KEY) eklenmiştir.

### 8.3 3. Aşama: Arayüz (Front-End) Tasarımı ve CSS Mimarisini

- **Design Token Mimari Kurgusu:** Tasarımın tüm renkleri ve tasarım değişkenleri CSS :root seçicisinde tanımlanarak tek merkezden yönetilebilir hale getirilmiştir.
- **Responsive Kodlama:** Mobil kullanıcıların kolayca bilet alabilmesi için CSS Media Queries kullanılmış, navbar elementlerine mobil uyumlu hamburger menüler entegre edilmiştir.

### 8.4 4. Aşama: Sunucu (Back-End) Programlama ve OOP Entegrasyonu

- **PDO Bağlantısı:** Veritabanına olan tüm erişimler güvenli PDO kütüphanesiyle yapılmış ve tüm sorgular Prepared Statements ile yapılandırılmıştır.
- **Controller Yapıları:** Sunucu tarafındaki form POST'ları, API istekleri ve yetki denetleyicileri bağımsız PHP dosyaları halinde geliştirilmiştir.

### 8.5 5. Aşama: Güvenlik, Hata Ayıklama (Debugging) ve Refactoring

- **Refactoring (Kod İyileştirme):** Proje kök dizininde yer alan kalabalık ve gereksiz dosyalar (eski setup dosyaları) temizlenmiş, giriş/kayıt kodları auth/ klasörü altına toplanarak yapı sadeleştirilmiştir.
- **Performans Optimizasyonu:** `common.css` içindeki sözdizimi hataları temizlenmiş, gereksiz arka plan gradient döngüleri kaldırılarak sayfa kaydırma performansı optimize edilmiştir.
- **Yetki Kontrolleri:** Firmaların yalnızca kendi ciro ve biletlerini yönetebilmesi için SQL sorgularındaki filtreler sıkılaştırılmıştır.

### 8.6 6. Aşama: Testler, Doğrulama ve Canlı Ortam Simülasyonu

- **Birim Testleri:** Bilet rezervasyonu kilit sürelerinin dolması ve kapasitelerin doğru şekilde geri aktarılması simüle edilerek kontrol edilmiştir.
- **Kullanıcı Deneyimi Testleri:** Ödeme modalı ve iade modalı test edilerek sunucudan gelen API yanıtlarının arayüze anlık yansımaları doğrulanmıştır.

## 9. PROJE MODÜLLERİNİN KOD YAPISI VE ANALİZİ

---

### 9.1 Güvenlik ve Yardımcı Fonksiyonlar (`includes/functions.php`)

Temel filtreleme ve veritabanı bağlantı işlemlerinin yer aldığı yardımcı dosya içeriğidir:

```

<?php
// includes/functions.php
require_once __DIR__ . '/../config/database.php';

function e($string) {
    return htmlspecialchars($string ?? '', ENT_QUOTES, 'UTF-8');
}

function base_url($path = '') {
    return '/ilterhoca/' . ltrim($path, '/');
}

function resolve_url($path) {
    if (empty($path)) {
        return 'https://placeholder.co/800x400/1a1a2e/7c3aed?text=Gorsel';
    }
    if (strpos($path, 'http') === 0) {
        return $path;
    }
    return base_url($path);
}

function expire_old_reservations($pdo) {
    try {
        $pdo->beginTransaction();
        // Süresi dolmuş rezervasyonları bul
        $stmt = $pdo->prepare(
            "SELECT id, event_id, ticket_count FROM reservations
            WHERE expires_at < NOW()"
        );
        $stmt->execute();
        $expired = $stmt->fetchAll(PDO::FETCH_ASSOC);

        if (!empty($expired)) {
            $delete = $pdo->prepare("DELETE FROM reservations WHERE id = ?");
            $restore = $pdo->prepare(
                "UPDATE events SET remaining_capacity = remaining_capacity + ?
                WHERE id = ?"
            );
            foreach ($expired as $res) {
                $delete->execute([$res['id']]);
                $restore->execute([$res['ticket_count'], $res['event_id']]);
            }
        }
        $pdo->commit();
    } catch (Exception $e) {
        $pdo->rollBack();
    }
}

```

## 9.2 Oturum ve Yetkilendirme Mantığı (includes/auth.php)

Kullanıcı giriş işlemlerinin ve yetki rollerinin sorgulandığı modüldür:



```

<?php
// includes/auth.php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

function start_secure_session() {
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }
}

function is_logged_in()    { return isset($_SESSION['user_id']); }
function is_superuser()   { return isset($_SESSION['role']) && $_SESSION['role']
=== 'admin'; }
function is_partner()      { return isset($_SESSION['role']) && $_SESSION['role']
=== 'partner'; }
function is_panel_user()   { return is_superuser() || is_partner(); }
function get_current_user_id() { return $_SESSION['user_id'] ?? null; }

```

### 9.3 İade İş Mantığı Fonksiyonları (includes/refund\_functions.php)

Etkinlik iptal ve iade süreçlerinin arka plandaki iş mantığı katmanıdır:

```

<?php
// includes/refund_functions.php

function get_all_refunds($pdo, $status = '', $limit = 20, $offset = 0, $partner_id
= null) {
    $sql = "SELECT r.*, u.name as full_name, u.email, e.title as event_title,
                t.order_code
            FROM refunds r
            JOIN users u    ON r.user_id    = u.id
            JOIN events e   ON r.event_id   = e.id
            JOIN tickets t  ON r.ticket_id  = t.id";

    $where = [];
    $params = [];

    if ($partner_id !== null) {
        $where[] = "e.created_by = ?";
        $params[] = $partner_id;
    }
    if (!empty($status)) {
        $where[] = "r.status = ?";
        $params[] = $status;
    }
    if (!empty($where)) {
        $sql .= " WHERE " . implode(" AND ", $where);
    }
    $sql .= " ORDER BY r.requested_at DESC LIMIT $limit OFFSET $offset";

    $stmt = $pdo->prepare($sql);
    $stmt->execute($params);
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}

function get_refund_statistics($pdo, $partner_id = null) {
    $stats = [
        'pending'    => ['count' => 0, 'total' => 0.0],
        'approved'   => ['count' => 0, 'total' => 0.0],
        'completed'  => ['count' => 0, 'total' => 0.0],
        'rejected'   => ['count' => 0, 'total' => 0.0]
    ];

    $sql = "SELECT r.status, COUNT(*) as count, SUM(r.refund_amount) as total
            FROM refunds r
            JOIN events e ON r.event_id = e.id";

    if ($partner_id !== null) {
        $sql .= " WHERE e.created_by = ?";
        $stmt = $pdo->prepare($sql . " GROUP BY r.status");
        $stmt->execute([$partner_id]);
    } else {
        $stmt = $pdo->prepare($sql . " GROUP BY r.status");
        $stmt->execute();
    }

    $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
    foreach ($rows as $row) {
        if (isset($stats[$row['status']])) {

```

```

        $stats[$row['status']] = [
            'count' => (int)$row['count'],
            'total' => (float)$row['total']
        ];
    }
}
return $stats;
}

```

## 9.4 İstemci Arayüzü - Etkinlik Arama ve Listeleme (events.php)

Etkinliklerin filtrelenebilir ve otomatik form submit (onchange) yöntemiyle listelenmesi sürecini yürüten kısımdır:

```

<?php
// events.php
define('ALLOWED_ACCESS', true);
require_once __DIR__ . '/includes/auth.php';
require_once __DIR__ . '/includes/functions.php';

start_secure_session();
expire_old_reservations($pdo);

$search = trim($_GET['search'] ?? '');
$city = trim($_GET['city'] ?? '');
$sort_by = trim($_GET['sort_by'] ?? 'date_asc');

$stmt = $pdo->prepare("SELECT DISTINCT city FROM events WHERE status = 'active'");
$stmt->execute();
$cities = array_column($stmt->fetchAll(PDO::FETCH_ASSOC), 'city');
?>

<!-- Filtreleme Paneli HTML Kısım -->
<form method="GET" class="events-filter-form">
    <input type="search" name="search" placeholder="Etkinlikleri ara..."
        value="<?= e($search) ?>" />
    <select name="city" onchange="this.form.submit()">
        <option value="">Tüm Şehirler</option>
        <?php foreach ($cities as $c): ?>
            <option value="<?= e($c) ?>" <?= $c === $city ? 'selected' : '' ?>>
                <?= e($c) ?>
            </option>
        <?php endforeach; ?>
    </select>
</form>

```

## 9.5 Bilet Satış ve Simüle POS Ödeme Ekranı (event.php)

Etkinlik bilet detaylarını getiren ve kullanıcıya bilet satışı simüle eden kontrol yapısıdır:

```
// event.php sayfasından bilet oluşturma controller mantığı
function purchase_ticket($pdo, $user_id, $event_id, $quantity, $amount) {
    $pdo->beginTransaction();
    try {
        $order_code = 'BG-' . strtoupper(bin2hex(random_bytes(5)));
        $stmt = $pdo->prepare(
            "INSERT INTO tickets (order_code, user_id, event_id, price_paid,
status)
            VALUES (?, ?, ?, ?, 'active')"
        );
        for ($i = 0; $i < $quantity; $i++) {
            $stmt->execute([$order_code, $user_id, $event_id, $amount]);
        }
        // Kullanıcıya sadakat puanı ver (Her 10 TL için 1 puan)
        $points = floor($amount * $quantity / 10);
        $updatePoints = $pdo->prepare(
            "UPDATE users SET loyalty_points = loyalty_points + ? WHERE id = ?"
        );
        $updatePoints->execute([$points, $user_id]);
        $pdo->commit();
        return $order_code;
    } catch (Exception $e) {
        $pdo->rollBack();
        return false;
    }
}
```

## 9.6 Yönetici Panelli İade Talebi API Yapısı (admin/api/manage\_refunds.php)

Yöneticinin Fetch API üzerinden gönderdiği POST isteğiyle iade taleplerini onaylayıp reddetmesini sağlayan RESTful API dosyasıdır:



```

<?php
// admin/api/manage_refunds.php
define('ALLOWED_ACCESS', true);
require_once __DIR__ . '/../../includes/auth.php';
require_once __DIR__ . '/../../includes/functions.php';

header('Content-Type: application/json');

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    echo json_encode(['success' => false, 'message' => 'Geçersiz istek metodu.']);
    exit;
}

$input      = json_decode(file_get_contents('php://input'), true);
$action     = $input['action'] ?? '';
$refund_id = (int)($input['refund_id'] ?? 0);

if (!$refund_id || !in_array($action, ['approve', 'reject', 'complete'])) {
    echo json_encode(['success' => false, 'message' => 'Geçersiz parametreler.']);
    exit;
}

try {
    if ($action === 'approve') {
        $stmt = $pdo->prepare("UPDATE refunds SET status = 'approved' WHERE id
= ?");
        $stmt->execute([$refund_id]);
        echo json_encode(['success' => true, 'message' => 'İade talebi
onaylandı.']);
    } elseif ($action === 'reject') {
        $reason = $input['rejection_reason'] ?? '';
        $stmt = $pdo->prepare(
            "UPDATE refunds SET status = 'rejected', rejection_reason = ? WHERE id
= ?"
        );
        $stmt->execute([$reason, $refund_id]);
        echo json_encode(['success' => true, 'message' => 'İade talebi
reddedildi.']);
    }
} catch (Exception $e) {
    echo json_encode(['success' => false, 'message' => $e->getMessage()]);
}

```

## 9.7 Yönetici İade Kontrol Sayfası Arayüzü (admin/manage\_refunds.php)

İade modalının ve tablolarının listelendiği ana sayfa arayüz elementleridir:

```

<!-- Modal Tetikleyici JS Fonksiyonu -->
<script>
function approveRefund() {
    const method = document.getElementById('refund-method').value;
    fetch('/ilterhoca/admin/api/manage_refunds.php', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({
            action: 'approve',
            refund_id: currentRefundId,
            refund_method: method
        })
    }).then(r => r.json()).then(data => {
        if (data.success) {
            alert(data.message);
            location.reload();
        }
    });
}
</script>

```

### 9.8 Kullanıcı Profil ve Bilet Yönetim Sayfası (my\_tickets.php)

Kullanıcının bilet iade işlemlerini kendisinin başlattığı ve geçmiş biletlerini listelediği modüldür. Bu sayfada kullanıcı aktif biletlerini görüntüleyebilir, etkinlik tarihine göre iade talep edebilir ve sipariş geçmişine erişebilir.

## 10. KARŞILAŞILAN ZORLUKLAR VE ÇÖZÜMLERİ

---

### 10.1 Eşzamanlı Satışlarda Yarış Durumu (Race Condition)

Popüler bir konser veya tiyatro etkinliğinde, son biletlerin satışı esnasında birden çok kullanıcının aynı milisaniye içerisinde UPDATE isteği göndermesi kapasite rakamlarının eksiye düşmesine sebep oluyordu. Bu durum, veri okuma anında satırı kilitleyen FOR UPDATE ifadesi ile çözülmüş ve sıradaki kullanıcıların beklemesi sağlanarak veri tutarlılığı güvence altına alınmıştır.

### 10.2 Arayüz Grafik Performansı ve GPU Optimizasyonu

CSS tabanlı arka plan radial gradyan animasyonlarının ve backdrop-filter: blur() cam tasarımlarının tarayıcı sayfa kaydırma (scroll) işlemlerinde ekran kartına aşırı yük bindirdiği tespit edilmiştir. Çözüm olarak, animasyonlar kaldırılmış ve GPU dostu düz alfa kanallı saydam arka plan kodlaması uygulanmıştır.

### 10.3 Veri Güvenliği ve İzolasyon Zafiyetleri

SQL sorgularında bulunan created\_by IS NULL parametresinin, firma hesaplarının sistem geneli diğer sahipsiz etkinlik ve biletleri görüntülemesine veya düzenlemesine yol açtığı keşfedilmiştir. Bu durum, sorgu parametrelerinden kaldırılarak firma id'si üzerinden katı bir eşleştirme kontrolüyle aşılmıştır.

## 11. PROJE EKLAN GÖRÜNTÜLERİ VE KULLANIM ADIMLARI

### [ EKLAN GÖRÜNTÜSÜ 1: ANA SAYFA VE ETKİNLİK VİTRİNİ ]

Kullanıcıların kategorilere göre dinamik arama yaptığı, güncel premium koyu tema arayüzüne sahip kartların listelendiği genel vitrin ekranıdır.

### [ EKLAN GÖRÜNTÜSÜ 2: DETAY VE REZERVASYON ADIMI ]

Etkinliğin kalan bilet sayısının anlık gösterildiği, bilet seçildiğinde 15 dakikalık sepet geri sayım sayacının devreye girdiği rezervasyon adımıdır.

### [ EKLAN GÖRÜNTÜSÜ 3: FİRMA VE ADMIN YÖNETİM PANELİ ]

Firma yetkililerinin kendi etkinliklerinin toplam cirosunu, sattıkları bilet adetlerini dinamik bilgi kartları (widget) üzerinden izlediği ve iade taleplerini onaylayıp reddettiği yönetim ekranıdır.



## 12. KAYNAKÇA

---

1. PHP Group. (2026). PHP Veritabanı Nesneleri (PDO) Güvenli Kodlama Kılavuzu. Erişim adresi: <https://www.php.net/manual/tr/book.pdo.php>
2. Oracle Corporation. (2026). MySQL İlişkisel Veritabanı ve Transaction Yönetimi El Kitabı. Erişim adresi: <https://dev.mysql.com/doc/>
3. Mozilla Developer Network (MDN). (2026). CSS Custom Properties (Design Tokens) Geliştirici Dokümanları.
4. OWASP Foundation. (2026). Cross-Site Request Forgery (CSRF) Önleme Hile Sayfası. Erişim adresi: <https://owasp.org/www-community/attacks/csrf>